

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND
COMMUNICATION TECHNOLOGY



PROJECT II – IT3930E

CIFAKE Image Classification with Explainable AI

Supervisor: ThS. Le Hai Nam

Student:

Tran Quang Trong – 20235565

Contents

1	Introduction	3
2	Dataset Analysis	5
2.1	Dataset Overview	5
2.2	Generation Process of FAKE Images	5
2.3	Dataset Structure	6
2.4	Train, Validation, and Test Split	6
2.5	Class Balance and Imbalance Analysis	7
2.6	Visual Characteristics of REAL and FAKE Images	7
2.7	Feature Characteristics Compared with Ordinary Image Classification	8
2.8	Challenges of the CIFAKE Dataset	9
3	Related Methods and Model Selection	11
3.1	Overview of Model Selection	11
3.2	SimpleCNN as a Baseline Model	12
3.3	ResNet18 and Residual Learning	13
3.4	EfficientNet-B0 and Model Efficiency	13
3.5	DenseNet121 and Dense Feature Reuse	14
3.6	DeiT-Tiny and Transformer-based Image Classification	15
3.7	Explainable AI with Grad-CAM	16
3.8	Summary of Model Roles	17
4	Experimental Setup	19
4.1	Preprocessing	19
4.2	Data Loading and Reproducible Split	19
4.3	Model Architecture Setup	19
4.3.1	SimpleCNN	20
4.3.2	ResNet18	21
4.3.3	EfficientNet-B0	21
4.3.4	DenseNet121	21
4.3.5	DeiT-Tiny	22
4.3.6	Summary of Architecture Adaptation	22
4.4	Model Output and Label Encoding	23
4.5	Loss Function	23
4.6	Hyperparameter Selection Strategy	24
4.7	Optimizer and Training Hyperparameters	25
4.8	Training Loop and Checkpoint Selection	27
4.9	Evaluation Protocol	27
4.10	Evaluation Metrics	29

5	Training Behavior Analysis	31
5.1	Training History and Analysis Method	31
5.2	SimpleCNN Training Behavior	31
5.3	ResNet18 Training Behavior	32
5.4	EfficientNet-B0 Training Behavior	33
5.5	DenseNet121 Training Behavior	34
5.6	DeiT-Tiny Training Behavior	35
5.7	Cross-Model Training Comparison	36
6	Experimental Results and Discussion	38
6.1	Final Test Results	38
6.2	Discussion of Model Performance	40
7	Explainability Analysis with Grad-CAM	41
7.1	Purpose and Scope of Explainability Analysis	41
7.2	Grad-CAM Method	41
7.3	Qualitative Grad-CAM Results	42
7.4	Quantitative Heatmap Analysis	43
7.5	Discussion of Attention Patterns	44
8	Gradio Demo	46
9	Limitations	48
10	Conclusion and Future Work	49
	References	51

Chapter 1

Introduction

The rapid development of generative artificial intelligence has made synthetic images increasingly realistic and difficult to distinguish from natural images. Modern image generation models can produce visually convincing images that may appear similar to real photographs. As a result, detecting whether an image is real or AI-generated has become an important computer vision problem, especially in contexts such as online media verification, digital content moderation, and trustworthy AI systems.

This project focuses on **REAL/FAKE image classification** using the CIFAKE dataset. CIFAKE is a binary image classification dataset containing real images and AI-generated synthetic images. The goal is to train deep learning models that can classify each input image into one of two classes: **REAL** or **FAKE**. However, this task is different from ordinary object classification. In standard image classification, a model usually learns semantic object features such as the shape of a car, the body of an animal, or the wings of an airplane. In CIFAKE, the object content between real and fake images can be visually similar, so the model must learn more subtle visual cues such as texture inconsistency, unnatural edges, background artifacts, color distribution, and local noise patterns.

Therefore, classification accuracy alone is not sufficient for this project. A model may achieve high accuracy but still rely on shortcut patterns, background bias, or irrelevant visual cues. For this reason, explainable AI is included as an important part of the study. Grad-CAM is used to visualize which image regions contribute most strongly to the model prediction. This allows the report to analyze whether the model focuses on meaningful artifact-like regions or whether it may depend on misleading patterns.

The project compares five model architectures: **SimpleCNN**, **ResNet18**, **EfficientNet-B0**, **DenseNet121**, and **DeiT-Tiny**. SimpleCNN is used as a baseline model to measure how much performance can be achieved with a simple convolutional neural network. ResNet18 represents residual learning, which helps deeper CNNs learn more stable visual representations. EfficientNet-B0 is included to evaluate the balance between classification performance and model efficiency. DenseNet121 is used because dense connections encourage feature reuse, which may be useful for detecting subtle texture-level artifacts. DeiT-Tiny is included as a Transformer-based model to compare CNN-based local feature extraction with patch-based self-attention.

The main objectives of this project are as follows:

- To analyze the CIFAKE dataset in terms of data structure, class balance, image resolution, and visual characteristics.
- To explain why REAL/FAKE image detection is different from ordinary image classification.

- To train and compare five models: SimpleCNN, ResNet18, EfficientNet-B0, DenseNet121, and DeiT-Tiny.
- To evaluate model performance using loss, accuracy, precision, recall, F1-score, and confusion matrix.
- To analyze training and validation curves in order to identify overfitting or underfitting.
- To use Grad-CAM and heatmap statistics to explain model predictions.
- To discuss why some models perform better than others based on both quantitative metrics and explainability results.

Overall, this project is designed not only to build an image classifier, but also to understand the behavior of different deep learning architectures on AI-generated image detection. The report follows a complete experimental workflow, including dataset analysis, preprocessing, model selection, training setup, performance evaluation, error analysis, explainability analysis, and final discussion.

Chapter 2

Dataset Analysis

2.1 Dataset Overview

The CIFAKE dataset is used in this project for binary classification between real images and AI-generated synthetic images. It contains two classes: **REAL** and **FAKE**. The REAL images are taken from the CIFAR-10 dataset, while the FAKE images are generated to create a synthetic counterpart with similar semantic categories.

The dataset contains 120,000 images in total. The training set contains 100,000 images, including 50,000 REAL images and 50,000 FAKE images. The test set contains 20,000 images, including 10,000 REAL images and 10,000 FAKE images. All images have a resolution of 32×32 pixels.

This dataset is suitable for studying AI-generated image detection because the two classes are balanced and visually similar at the semantic level. The task is therefore not simply to recognize object categories, but to detect whether the image comes from a natural image distribution or a synthetic image generation process.

2.2 Generation Process of FAKE Images

An important characteristic of CIFAKE is how the FAKE images are produced. The FAKE images are not randomly collected from the Internet. Instead, they are generated to mirror the semantic categories of CIFAR-10. Since CIFAR-10 contains ten object categories, the FAKE subset is designed to contain synthetic images that correspond to similar types of visual content.

In the CIFAKE dataset, REAL images come from CIFAR-10, while FAKE images are generated using Stable Diffusion version 1.4. This means that the FAKE images are produced by a generative model rather than captured by a camera. As a result, the two classes may contain similar objects, such as vehicles, animals, airplanes, or ships, but they differ in image origin.

This generation process is important for the design of the experiment. If REAL and FAKE images share similar object categories, the model should not solve the task by simply learning object-category differences. Instead, it needs to learn more subtle visual cues that distinguish real images from synthetic images. These cues may include texture inconsistency, unnatural edges, background artifacts, local distortions, color distribution differences, or noise patterns introduced by the generation process.

However, this also introduces a limitation. Since the FAKE images are generated using a specific image generation model, a classifier may learn generator-specific traces rather

than general synthetic-image features. Therefore, strong performance on CIFAKE does not automatically guarantee that the same model will generalize to all AI-generated images from newer or different generators. This issue should be considered when interpreting the final results.

2.3 Dataset Structure

The dataset is organized into separate folders for training and testing. Each split contains two class folders: **REAL** and **FAKE**. The structure used in this project is shown below:

```
dataset/
|-- train/
|   |-- REAL/
|   |-- FAKE/
|
|-- test/
|   |-- REAL/
|   |-- FAKE/
```

This folder organization is convenient for image classification tasks because each subfolder directly represents a class label. During data loading, images inside the REAL folder are assigned the REAL label, while images inside the FAKE folder are assigned the FAKE label.

The dataset statistics are summarized in Table 2.1.

Table 2.1: CIFAKE dataset statistics		
Split	Class	Number of Images
Train	REAL	50,000
Train	FAKE	50,000
Test	REAL	10,000
Test	FAKE	10,000

From this structure, the original training set contains 100,000 images and the original test set contains 20,000 images. The dataset is balanced in both splits because each split contains the same number of REAL and FAKE images.

2.4 Train, Validation, and Test Split

The original CIFAKE training set is further divided into training and validation subsets. In this project, 80% of the original training set is used for training, and 20% is used for validation. The original test set is kept unchanged and is used only for final evaluation.

The resulting experimental split is shown in Table 2.2.

Table 2.2: Train, validation, and test split used in this project

Subset	REAL	FAKE	Total	Purpose
Training	40,000	40,000	80,000	Model learning
Validation	10,000	10,000	20,000	Model selection and early stopping
Test	10,000	10,000	20,000	Final evaluation

This protocol avoids using the test set during training or model selection. The validation set is used to monitor model performance during training and to select the best checkpoint. The test set is only used after training is complete, which provides a fairer estimate of final model performance.

2.5 Class Balance and Imbalance Analysis

Class imbalance is an important issue in classification tasks because a model may achieve high accuracy by favoring the majority class. In CIFAKE, however, the number of REAL and FAKE images is equal in the training set, validation set, and test set. Therefore, the dataset does not suffer from class imbalance at the binary label level.

The balanced class distribution has two main implications. First, accuracy is a meaningful metric because the model cannot obtain high performance by simply predicting the majority class. Second, even though accuracy is useful, it is still necessary to analyze precision, recall, F1-score, and confusion matrix. These metrics reveal whether the model makes different types of errors, such as predicting REAL images as FAKE or predicting FAKE images as REAL.

For this problem, the error type **FAKE predicted as REAL** is especially important. This is a false negative in fake image detection. In practical applications, this type of error may be more harmful because a synthetic image is incorrectly accepted as real. Therefore, recall for the FAKE class is an important metric when evaluating the reliability of the model.

Although the REAL/FAKE labels are balanced, this does not guarantee that the dataset is completely free from bias. There may still be differences in object categories, background patterns, color distributions, or generator-specific artifacts. Therefore, the report should not only check class balance, but also discuss visual characteristics and possible shortcut learning.

2.6 Visual Characteristics of REAL and FAKE Images

Visual inspection is useful for understanding the difficulty of the CIFAKE classification task. Figure 2.1 shows example images from the CIFAKE dataset. The samples illustrate that REAL and FAKE images may contain similar semantic content, such as vehicles, animals, or other CIFAR-10-like objects. Because the images are only 32×32 pixels, many details are compressed or blurred, making the difference between real and synthetic images difficult to identify by visual inspection alone.

CIFAKE Dataset Samples



Figure 2.1: Example REAL and FAKE images from the CIFAKE dataset.

The visual similarity between the two classes shows that CIFAKE is not a simple object recognition task. A model cannot rely only on whether an image contains a truck, car, bird, cat, deer, or dog, because similar object categories may appear in both REAL and FAKE subsets. Instead, the model needs to learn more subtle differences related to the origin of the image.

Possible visual cues include:

- unnatural texture patterns,
- inconsistent object boundaries,
- unusual background regions,
- local distortions,
- over-smoothed or noisy regions,
- color distribution differences,
- edge artifacts introduced by the generative process.

These visual differences may be difficult to observe directly, but deep learning models can learn statistical patterns from large numbers of examples. This is why model evaluation and explainability analysis are both necessary. Evaluation metrics show whether the model can classify the images correctly, while explainability methods help inspect which regions the model uses to make decisions.

2.7 Feature Characteristics Compared with Ordinary Image Classification

CIFAKE classification is different from ordinary object classification. In a standard image classification problem, the label is usually determined by the object category. For example, a model may classify an image as a car, cat, dog, airplane, or ship. The model mainly learns semantic object features such as shape, parts, and spatial structure.

In CIFAKE, the label is not determined by what object appears in the image. Instead, the label is determined by the image origin: whether the image is real or AI-generated.

This makes the problem more subtle. A REAL image and a FAKE image may both contain the same type of object, but they belong to different classes because they come from different sources.

Table 2.3 summarizes this difference.

Table 2.3: Comparison between ordinary object classification and CIFAKE classification

Ordinary Object Classification	CIFAKE REAL/FAKE Classification
The label depends on object category.	The label depends on image origin.
The model learns semantic object features.	The model learns real-versus-synthetic visual cues.
Object regions are usually the most important.	Background, texture, and artifacts may also be important.
Examples include cat, dog, car, airplane, and ship.	Examples include REAL and FAKE.
Main cues include shape, parts, and object structure.	Main cues may include texture, edge, noise, and local artifacts.

This distinction affects model selection and interpretation. CNN-based models may be effective because convolutional filters are naturally good at extracting local texture, edge, and spatial patterns. DenseNet may be particularly useful because dense connections reuse low-level and mid-level features, which may help detect subtle artifacts. Transformer-based models such as DeiT-Tiny provide a different perspective by using patch-level self-attention, but they may require more careful tuning for low-resolution images.

2.8 Challenges of the CIFAKE Dataset

The CIFAKE dataset presents several challenges for real-versus-synthetic image detection.

First, the image resolution is very low. Since each image is only 32×32 pixels, fine-grained details are limited. This makes both classification and explanation more difficult. Grad-CAM visualizations may also become coarse because the heatmap is generated from a low-resolution input.

Second, REAL and FAKE images can have similar semantic content. Since the FAKE images are generated to mirror CIFAR-10 categories, the model cannot simply rely on object category. It must learn visual differences related to image origin.

Third, synthetic artifacts may be subtle and distributed across the image. They may not appear only on the main object. Instead, useful evidence may appear in background regions, edges, texture patterns, or small local distortions. This makes explainability analysis important because it helps determine where the model is looking.

Fourth, there is a risk of shortcut learning. A model may learn dataset-specific or generator-specific patterns instead of general fake-image features. For example, if Stable Diffusion-generated images contain repeated visual traces, the model may rely on these traces. This can lead to strong performance on CIFAKE but weaker generalization to images generated by other models.

Finally, because the dataset only provides binary labels, it is not possible to directly analyze different types of synthetic images by generator, prompt, or generation setting.

This limits the depth of generator-specific analysis and should be considered as a limitation of the project.

Chapter 3

Related Methods and Model Selection

3.1 Overview of Model Selection

The goal of this project is not only to train a single classifier, but also to compare different deep learning architectures for the CIFAKE REAL/FAKE image classification task. Since CIFAKE is different from ordinary object classification, the selected models should represent different ways of extracting visual features from images.

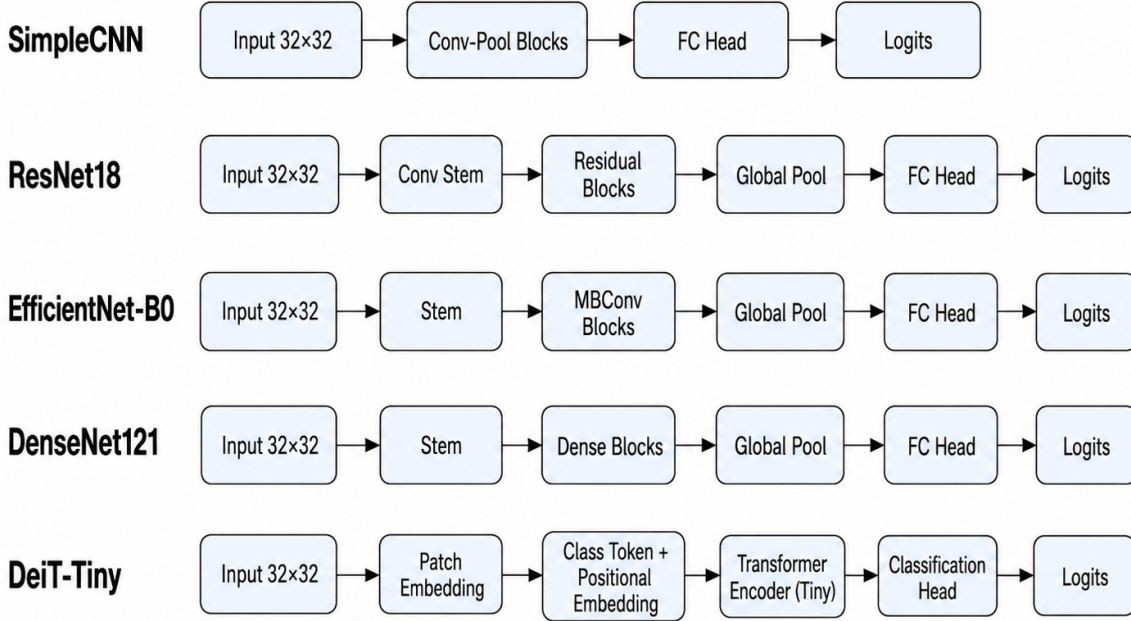
In this project, five models are selected: **SimpleCNN**, **ResNet18**, **EfficientNet-B0**, **DenseNet121**, and **DeiT-Tiny**. These models are not chosen randomly. Each model plays a different role in the experimental comparison.

SimpleCNN is used as a baseline model. It shows how much performance can be achieved by a simple convolutional neural network without advanced architectural design. ResNet18 represents residual learning, which helps deeper networks learn more stable visual representations. EfficientNet-B0 represents efficient model scaling, where the architecture is designed to balance accuracy and computational cost. DenseNet121 represents dense feature reuse, which may be useful for detecting subtle texture-level artifacts. DeiT-Tiny represents a Transformer-based approach, which uses patch embeddings and self-attention instead of only convolutional feature extraction.

This model selection allows the report to compare not only performance numbers, but also architectural behavior. For example, CNN-based models are expected to be strong at capturing local patterns such as edges, textures, and small artifacts. In contrast, DeiT-Tiny uses self-attention to model relationships between image patches, which may help capture broader image-level inconsistencies but may also require more careful tuning for low-resolution images.

Figure 3.1 presents a high-level comparison of the five model architectures used in this project.

High-Level Model Architecture Comparison



Note: Blocks represent high-level processing stages rather than full layer-by-layer implementations.

Figure 3.1: High-level architecture comparison of SimpleCNN, ResNet18, EfficientNet-B0, DenseNet121, and DeiT-Tiny.

The figure does not describe every layer in detail. Instead, it summarizes the main processing stages of each model. This is useful because the models have different internal structures, but they all follow the same general classification objective: transform an input image into a final REAL/FAKE prediction.

3.2 SimpleCNN as a Baseline Model

SimpleCNN is used as the baseline architecture in this project. A baseline model is important because it provides a reference point for evaluating more complex architectures. Without a baseline, it would be difficult to know whether the performance improvement of deeper models is truly meaningful.

The SimpleCNN architecture consists of convolutional blocks, activation functions, pooling layers, and fully connected layers. Convolutional layers extract local image patterns, pooling layers reduce spatial resolution, and fully connected layers perform final classification based on the extracted features.

For CIFAKE, SimpleCNN is useful because it tests whether local low-level visual cues are already sufficient to distinguish REAL and FAKE images. If SimpleCNN achieves reasonable performance, it suggests that the dataset contains learnable differences at the pixel, texture, or local pattern level. These differences may include color statistics, edge irregularities, local noise patterns, or simple texture artifacts.

However, SimpleCNN also has limitations. Because it is relatively shallow, it may

not capture complex hierarchical representations. Subtle synthetic-image artifacts may require deeper feature extraction, especially when the difference between REAL and FAKE images is not visually obvious. Therefore, SimpleCNN is not expected to be the strongest model, but it is very important for understanding the difficulty of the task.

In this project, SimpleCNN plays the following role:

- It provides a simple baseline for comparison.
- It tests whether low-level visual patterns are useful for CIFAKE detection.
- It helps measure the benefit of using deeper or more advanced architectures.
- It provides a lightweight model that is easier to train and interpret.

3.3 ResNet18 and Residual Learning

ResNet18 is selected as a representative residual convolutional neural network. The key idea of ResNet is the use of residual connections, also known as skip connections. Instead of forcing each block to learn a complete transformation, residual learning allows a block to learn a residual mapping. This helps information and gradients flow more easily through the network.

This design is important for training deeper CNNs. In traditional deep networks, adding more layers can sometimes make training more difficult because of vanishing gradients or degradation in optimization. Residual connections reduce this problem by allowing earlier information to pass directly to later layers.

For CIFAKE classification, ResNet18 is useful because it can learn deeper hierarchical features than SimpleCNN. Early layers can detect simple visual patterns such as edges and textures, while deeper layers can combine these patterns into more abstract representations. This may help the model identify subtle differences between real and synthetic image distributions.

ResNet18 is also a reasonable model choice because it is not extremely large. It provides a balance between model depth and training cost. This makes it suitable for comparison with both simpler models such as SimpleCNN and more complex models such as DenseNet121.

In this project, ResNet18 plays the following role:

- It represents residual CNN-based feature learning.
- It tests whether deeper hierarchical features improve REAL/FAKE classification.
- It provides a stronger CNN baseline than SimpleCNN.
- It helps evaluate the benefit of skip connections in detecting synthetic-image artifacts.

3.4 EfficientNet-B0 and Model Efficiency

EfficientNet-B0 is selected to represent efficient convolutional neural network design. Unlike models that simply increase depth or width, EfficientNet is based on the idea of

compound scaling. This means that the model balances network depth, width, and input resolution in a systematic way.

The motivation for including EfficientNet-B0 is that classification performance should not be evaluated only by accuracy. In practical applications, a model may also need to be efficient in terms of parameter count, memory usage, and inference speed. A model with slightly lower accuracy but much lower computational cost may be more suitable for deployment.

For CIFAKE, EfficientNet-B0 is useful because the input images are low-resolution and the classification target is binary. A very large model may not always be necessary. EfficientNet-B0 tests whether a compact and efficient architecture can still capture enough visual evidence to distinguish REAL from FAKE images.

EfficientNet-B0 uses mobile inverted bottleneck convolution blocks, commonly referred to as MBConv blocks. These blocks are designed to extract useful features while keeping computation efficient. This makes EfficientNet-B0 a strong candidate when both accuracy and efficiency are important.

In this project, EfficientNet-B0 plays the following role:

- It represents efficient CNN architecture design.
- It tests the trade-off between accuracy and computational efficiency.
- It evaluates whether a compact model can perform competitively on CIFAKE.
- It provides a practical comparison against larger or more feature-rich CNNs.

3.5 DenseNet121 and Dense Feature Reuse

DenseNet121 is selected because it uses dense connections between layers. In a DenseNet block, each layer receives feature maps from all previous layers. This design encourages feature reuse and improves gradient flow throughout the network.

Dense feature reuse is particularly relevant to CIFAKE. In REAL/FAKE image detection, useful cues may not only appear in high-level semantic regions. They may also appear in low-level or mid-level visual patterns, such as texture irregularities, edge inconsistencies, background artifacts, and local distortions. A model that can preserve and reuse earlier feature maps may be better at detecting these subtle cues.

Compared with ResNet, which uses residual addition, DenseNet concatenates feature maps from previous layers. This allows later layers to access a richer collection of features. For CIFAKE, this may help combine low-level texture information with deeper semantic representations.

DenseNet121 is therefore expected to perform well when the classification task depends on subtle visual evidence distributed across different feature levels. It is not only a deeper CNN, but also a model with a feature reuse mechanism that may be suitable for artifact detection.

In this project, DenseNet121 plays the following role:

- It represents dense feature reuse in CNNs.
- It tests whether low-level and mid-level feature preservation helps CIFAKE detection.

- It is expected to capture subtle texture and artifact patterns effectively.
- It serves as one of the strongest CNN-based candidates in the comparison.

3.6 DeiT-Tiny and Transformer-based Image Classification

DeiT-Tiny, or Data-efficient Image Transformer, is included as the Transformer-based model in this project. Unlike CNNs, DeiT-Tiny does not rely primarily on convolutional filters. Instead, it divides an image into patches, converts these patches into embeddings, and processes them using Transformer encoder blocks with self-attention.

The main reason for including DeiT-Tiny is to compare CNN-based local feature extraction with Transformer-based patch-level attention. CNNs have a strong local spatial inductive bias. This means they naturally focus on nearby pixels, local textures, edges, and small spatial patterns. This property may be very useful for CIFAKE, because synthetic-image artifacts can appear as local texture or edge abnormalities.

DeiT-Tiny, on the other hand, models relationships between image patches. Self-attention allows the model to connect distant patches and capture broader global relationships. This may be useful if the difference between REAL and FAKE images appears as global inconsistency across the image rather than only local artifacts.

However, DeiT-Tiny may also be more difficult to train in this project. CIFAKE images are only 32×32 pixels, which means there is limited spatial information. Transformer-based models often require careful training strategies, such as suitable patch size, learning rate scheduling, weight decay, data augmentation, and sometimes pretrained weights. Without careful tuning, DeiT-Tiny may overfit or generalize worse than CNN-based models.

Figure 3.2 summarizes the main difference between CNN-based feature extraction and DeiT-Tiny-based Transformer feature extraction.

CNN vs Transformer Feature Extraction

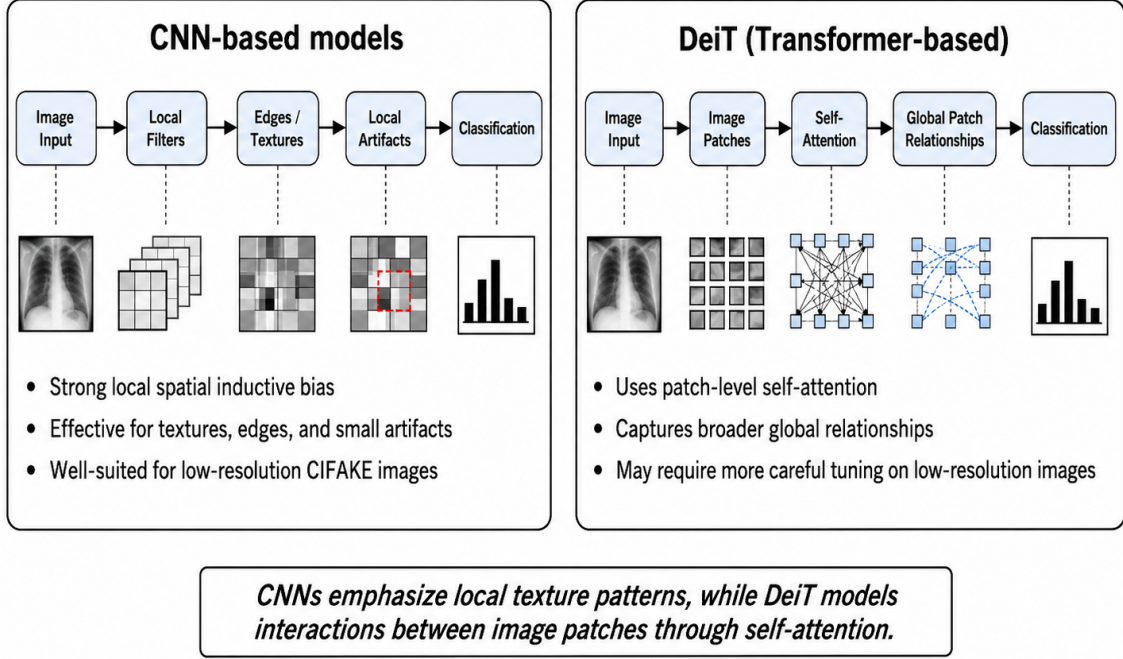


Figure 3.2: Comparison between CNN-based local feature extraction and DeiT-Tiny Transformer-based patch attention.

This comparison is important for interpreting the final results. If CNN models outperform DeiT-Tiny, one possible explanation is that local texture and artifact detection are more important for CIFAKE than global patch relationships. If DeiT-Tiny performs well, it may suggest that patch-level attention can capture useful global inconsistencies in synthetic images. Therefore, DeiT-Tiny is included not only as another model, but also as a different architectural perspective.

In this project, DeiT-Tiny plays the following role:

- It represents Transformer-based image classification.
- It compares self-attention with convolutional feature extraction.
- It tests whether global patch relationships help detect AI-generated images.
- It highlights the difference between CNN inductive bias and Transformer flexibility.

3.7 Explainable AI with Grad-CAM

Explainable AI is an important part of this project because high accuracy alone does not fully explain model behavior. A model may classify images correctly, but it may rely on irrelevant regions, background bias, or generator-specific shortcuts. Therefore, explainability is needed to inspect how the model makes decisions.

Grad-CAM, or Gradient-weighted Class Activation Mapping, is used to visualize important image regions for CNN-based models. It works by using gradients from the target class score with respect to convolutional feature maps. These gradients indicate how important each feature map is for the final prediction. The weighted feature maps are then combined to produce a heatmap that highlights the regions contributing most to the model output.

For CIFAKE, Grad-CAM is useful because the task is not ordinary object recognition. In normal object classification, it is often expected that the model focuses on the main object. In REAL/FAKE detection, however, important evidence may appear in background regions, textures, edges, or small artifact-like areas. Therefore, Grad-CAM helps determine whether the model is focusing on reasonable visual cues.

Grad-CAM is especially suitable for CNN models such as SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121 because these models contain convolutional feature maps. For DeiT-Tiny, Grad-CAM is not directly comparable in the same way because DeiT-Tiny is based on patch embeddings and Transformer self-attention. If Transformer explanations are needed, methods such as attention visualization or attention rollout may be more appropriate.

In this project, Grad-CAM plays the following role:

- It provides visual explanations for CNN-based predictions.
- It helps inspect whether models focus on object regions, background, texture, or artifacts.
- It supports qualitative analysis of correct and incorrect predictions.
- It helps identify possible shortcut learning or unreliable attention patterns.

3.8 Summary of Model Roles

Table 3.1 summarizes the purpose of each model in this project. This table helps clarify that each architecture is included for a specific reason, rather than simply being used as another classifier.

Table 3.1: Summary of model roles in the CIFAKE experiment

Model	Architecture Type	Role in This Project
SimpleCNN	Basic convolutional neural network	Serves as a baseline to measure how much performance can be achieved with a simple CNN.
ResNet18	Residual CNN	Tests whether residual learning and deeper hierarchical features improve REAL/FAKE detection.
EfficientNet-B0	Efficient CNN	Evaluates the trade-off between classification performance and computational efficiency.
DenseNet121	Dense CNN	Tests whether dense feature reuse helps detect subtle texture and artifact-level cues.
DeiT-Tiny	Transformer-based image classifier	Compares CNN local feature extraction with patch-based self-attention and global relationships.

Overall, the selected models provide a structured comparison between simple CNNs, deeper CNNs, efficient CNNs, dense feature-reuse CNNs, and Transformer-based image classification. This makes the experimental analysis more meaningful because the final results can be interpreted based on architectural differences, not only based on accuracy values.

Chapter 4

Experimental Setup

4.1 Preprocessing

Purpose: Explain how input images are prepared before being passed into the models.

Main points:

- Images are loaded using PIL and converted to RGB.
- The transform pipeline uses only `ToTensor()`.
- Pixel values are scaled from $[0, 255]$ to $[0, 1]$.
- No resizing is applied because CIFAKE images are already 32×32 .

4.2 Data Loading and Reproducible Split

Purpose: Explain how the dataset is loaded and split fairly.

Main points:

- Dataset directory structure: `dataset/train` and `dataset/test`.
- Binary label mapping: `FAKE = 0`, `REAL = 1`.
- Original training set is split into training and validation subsets.
- Validation ratio is 0.2, so training uses 80
- The split is performed separately by label to preserve class balance.
- Random seed is 42 for reproducibility.
- Training loader uses shuffle; validation and test loaders do not.

4.3 Model Architecture Setup

This section describes how the model architectures are adapted for the binary classification task on the CIFAKE dataset. The dataset contains two classes: `REAL` and `FAKE`. Instead of designing the models to output two class scores, all models in this project are modified to output only one logit. This design is suitable for binary classification with

`BCEWithLogitsLoss`, where the single output value is later converted into a probability by the sigmoid function.

Let x be an input image and let z be the model output. Each model is designed to learn a function:

$$f_{\theta}(x) = z,$$

where $z \in \mathbb{R}$ is a single logit and θ represents the trainable parameters of the model. A larger value of z indicates that the model is more confident that the image belongs to the positive class, which is **REAL** in this project. A smaller value of z indicates that the model is more confident that the image belongs to the negative class, which is **FAKE**.

4.3.1 SimpleCNN

The SimpleCNN model is used as a custom baseline architecture. It is simpler than the other deep learning models and is included to provide a basic comparison against larger architectures such as ResNet18, EfficientNet-B0, DenseNet121, and DeiT-Tiny.

The feature extractor of SimpleCNN contains two convolutional blocks. The first convolutional layer receives a three-channel RGB image as input and produces 32 feature maps:

$$\text{Conv2d}(3, 32, 3, \text{padding} = 1).$$

The kernel size is 3×3 , and padding is set to 1 so that the spatial size is preserved after convolution. After this convolution, a ReLU activation is used to introduce non-linearity, followed by max pooling to reduce the spatial resolution.

The second convolutional layer keeps the number of channels at 32:

$$\text{Conv2d}(32, 32, 3, \text{padding} = 1).$$

This layer extracts deeper image features from the previous feature maps. It is also followed by ReLU activation and max pooling. Since the CIFAKE images have size 32×32 , applying max pooling twice reduces the spatial resolution from 32×32 to 16×16 , and then to 8×8 .

Therefore, before entering the classifier, the feature tensor has the shape:

$$32 \times 8 \times 8.$$

The classifier first flattens this tensor into a vector and then applies a fully connected layer:

$$\text{Linear}(32 \times 8 \times 8, 64).$$

This layer maps the extracted image features into a 64-dimensional hidden representation. A ReLU activation is then applied, and the final linear layer produces one output logit:

$$\text{Linear}(64, 1).$$

Thus, the SimpleCNN model can be summarized as:

RGB Image $\rightarrow$ Convolutional Feature Extractor $\rightarrow$ Flatten $\rightarrow$ Fully Connected Classifier $\rightarrow$ One Logit

4.3.2 ResNet18

ResNet18 is a residual convolutional neural network. Its main advantage is the use of residual connections, which help the model learn deeper representations more effectively. In the original ResNet18 architecture, the final fully connected layer is designed for multi-class classification, such as ImageNet classification with 1000 classes.

In this project, ResNet18 is initialized with:

```
weights=None.
```

This means that pretrained ImageNet weights are not loaded. The model is trained from scratch using only the CIFAKE training dataset.

To adapt ResNet18 for binary classification, its original final fully connected layer, named `fc`, is replaced by a new linear layer with one output unit:

```
Linear(in_features,1).
```

Here, `in_features` is taken from the original ResNet18 classification head. This ensures that the new classifier receives the correct number of input features from the ResNet backbone while changing the output dimension to one.

Therefore, the adapted ResNet18 model produces one logit instead of a vector of class scores.

4.3.3 EfficientNet-B0

EfficientNet-B0 is a convolutional architecture designed to balance model depth, width, and input resolution efficiently. Compared with a simple CNN, it can learn more expressive image representations while still keeping the model relatively compact.

In this project, EfficientNet-B0 is also initialized with:

```
weights=None.
```

Therefore, the model does not use pretrained ImageNet parameters. All weights are learned directly from the CIFAKE dataset.

The original EfficientNet-B0 classifier is stored as a sequential module. In the implementation, the final classifier layer at index 1 is replaced:

```
model.classifier[1] = Linear(in_features,1).
```

This modification keeps the EfficientNet-B0 feature extraction backbone but changes the final prediction layer so that it outputs a single logit for binary classification.

The resulting output can be interpreted in the same way as the other models: after applying the sigmoid function, the logit becomes the predicted probability that the image is REAL.

4.3.4 DenseNet121

DenseNet121 is another convolutional neural network architecture. Its key characteristic is dense connectivity, where layers receive feature information from earlier layers. This helps feature reuse and can improve gradient flow during training.

The DenseNet121 model is initialized with:

`weights=None`.

This means the model is trained from scratch rather than fine-tuned from pretrained ImageNet weights.

In the original DenseNet121 architecture, the final classifier is designed for multi-class classification. For the CIFAKE binary classification task, this classifier is replaced by:

`Linear(in_features, 1)`.

Here, `in_features` is obtained from the original DenseNet121 classifier. The purpose is to preserve the feature dimension produced by the DenseNet backbone while changing the number of output units to one.

As a result, DenseNet121 also outputs one logit for each input image.

4.3.5 DeiT-Tiny

DeiT-Tiny is a lightweight vision transformer model. Unlike convolutional neural networks, which extract features using convolutional filters, DeiT processes an image as a sequence of patches. This allows the model to learn relationships between different image regions through self-attention.

In this project, DeiT-Tiny is created using the `timm` library with the model name:

`deit_tiny_patch16_224`.

Although the original DeiT-Tiny architecture is commonly associated with larger ImageNet-style input images, the implementation adapts it to the CIFAKE image size by setting:

`img_size=32`.

This indicates that the input images have spatial size 32×32 . The model also uses:

`in_chans=3`,

which means that the input images contain three color channels: red, green, and blue. Finally, the output dimension is set by:

`num_classes=1`.

This makes DeiT-Tiny output one logit instead of multiple class scores.

The DeiT-Tiny class also includes a `pretrained` argument, whose default value is `False`. Therefore, unless explicitly changed, the DeiT-Tiny model is also trained from scratch in this project.

4.3.6 Summary of Architecture Adaptation

Although the five models have different internal architectures, they are adapted using the same binary-output principle. SimpleCNN directly defines its final layer as `Linear(64, 1)`. ResNet18 replaces its `fc` layer. EfficientNet-B0 replaces `classifier[1]`. DenseNet121 replaces its `classifier`. DeiT-Tiny sets `num_classes=1` when the model is created.

The common output format for all models is:

$$z \in \mathbb{R}^{B \times 1},$$

where B is the batch size. This output shape is consistent with the target label shape used during training. Since each image has only one binary label, one logit per image is sufficient.

All models in this project are trained from scratch. This setting avoids using external pretrained ImageNet knowledge and makes the comparison focus on how well each architecture learns from the CIFAKE dataset itself.

4.4 Model Output and Label Encoding

In this binary classification problem, the model does not directly output the class label. Instead, the final layer produces a real-valued score called a logit, denoted by z . The logit can take any value in the interval $(-\infty, +\infty)$.

To convert this logit into a probability, the sigmoid function is applied:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where p represents the predicted probability that the input image belongs to the positive class. In this project, the positive class is encoded as `REAL = 1`, while the negative class is encoded as `FAKE = 0`. Therefore, a larger value of p means that the model is more confident that the image is real.

The predicted class label is obtained by applying a threshold of 0.5:

$$\hat{y} = \begin{cases} 1, & \text{if } p \geq 0.5, \\ 0, & \text{if } p < 0.5. \end{cases}$$

Because `REAL` is encoded as 1 and `FAKE` is encoded as 0, the meaning of the prediction is:

$$\hat{y} = 1 \Rightarrow \text{REAL}, \quad \hat{y} = 0 \Rightarrow \text{FAKE}.$$

During training, the label tensor is converted to floating-point format because the loss function computes continuous probability-based errors, not integer class indices. The labels are also reshaped using `unsqueeze(1)` so that their shape matches the model output shape. For example, if the model outputs a tensor of shape $[B, 1]$, where B is the batch size, the labels must also have shape $[B, 1]$ instead of $[B]$.

4.5 Loss Function

Since the task is binary classification, binary cross-entropy is used as the loss function. For one training sample, let $y \in \{0, 1\}$ be the ground-truth label and let p be the predicted probability after applying the sigmoid function. The binary cross-entropy loss is defined as:

$$\mathcal{L}_{\text{BCE}}(y, p) = -[y \log(p) + (1 - y) \log(1 - p)].$$

For a batch of N samples, the average binary cross-entropy loss is:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

The meaning of this loss can be understood from the two possible label values. If the true label is $y = 1$, meaning the image is real, the loss becomes:

$$\mathcal{L}_{\text{BCE}}(1, p) = -\log(p).$$

In this case, the model is penalized heavily when p is small, because a small p means the model incorrectly believes that the image is fake.

If the true label is $y = 0$, meaning the image is fake, the loss becomes:

$$\mathcal{L}_{\text{BCE}}(0, p) = -\log(1 - p).$$

In this case, the model is penalized heavily when p is large, because a large p means the model incorrectly believes that the image is real.

In the implementation, `BCEWithLogitsLoss` is used instead of manually applying sigmoid followed by binary cross-entropy. This is because `BCEWithLogitsLoss` combines the sigmoid operation and the binary cross-entropy computation into one numerically stable function. It works directly with the logit z :

$$\mathcal{L}(y, z) = -[y \log(\sigma(z)) + (1 - y) \log(1 - \sigma(z))].$$

Internally, this combined form avoids numerical instability when the logit z is very large or very small. Therefore, it is safer and more stable than computing `sigmoid` and then applying BCE separately.

4.6 Hyperparameter Selection Strategy

- Hyperparameters are selected using two principles:
 - controlled comparison,
 - architecture-specific adjustment.
- CNN models share the same setup because they are all convolution-based and trained from scratch.
- Shared CNN setup makes comparison fairer because performance differences are more likely due to architecture.
- However, shared CNN parameters are a baseline, not necessarily optimal for each CNN.
- DeiT-Tiny uses a different setup because Transformer-based models have different optimization behavior.

4.7 Optimizer and Training Hyperparameters

The optimizer and training hyperparameters are selected according to the characteristics of each model group. In this project, SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121 are treated as convolution-based models, while DeiT-Tiny is treated separately because it is a vision transformer architecture. Although all models solve the same binary classification problem, they do not have the same training behavior. Therefore, the hyperparameter settings are not chosen identically for all architectures. For SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121, the Adam optimizer is used with a learning rate of 0.001. Adam is selected because it adapts the learning rate for each parameter based on estimates of the first and second moments of the gradient. This makes training more stable than standard stochastic gradient descent, especially when training different architectures from scratch. Since the models are initialized without pretrained ImageNet weights, the optimizer must learn useful image representations directly from the CIFAKE training data. Adam is suitable in this setting because it can converge quickly during the early training stages. The learning rate is set to:

$$\eta = 0.001.$$

This value is a common and practical starting point for Adam-based training. It is large enough to allow the model parameters to update effectively, but not too large to make the optimization process unstable. For the convolution-based models, this learning rate provides a balance between convergence speed and training stability. A much larger learning rate could cause the loss to fluctuate or diverge, while a much smaller learning rate could make training unnecessarily slow. The batch size for these models is set to:

$$B = 64.$$

A batch size of 64 provides a reasonable trade-off between computational efficiency and gradient stability. A very small batch size may produce noisy gradient estimates, which can make the training curve unstable. A very large batch size may require more memory and may reduce the number of parameter updates per epoch. Since the convolution-based models can process CIFAKE images efficiently, a batch size of 64 is suitable for using the hardware resources effectively while still maintaining stable optimization. The maximum number of training epochs is set to 50:

$$E_{\max} = 50.$$

This value gives the models enough training time to learn meaningful visual features from the dataset. However, the models are not forced to train for all 50 epochs. Early stopping is used to stop training when the validation performance does not improve for several consecutive epochs. For these convolution-based models, the patience value is set to:

$$\text{patience} = 5.$$

This means that if the validation loss does not improve for 5 consecutive epochs, the training process is stopped. The purpose of this setting is to reduce overfitting and avoid wasting computation after the model has already reached its best validation performance. A patience value of 5 is strict enough to prevent unnecessary training, while still allowing the model some time to recover from temporary fluctuations in validation loss. Therefore,

the training configuration for SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121 is:

$$\text{optimizer} = \text{Adam}, \quad \eta = 0.001, \quad B = 64, \quad E_{\max} = 50, \quad \text{patience} = 5.$$

For DeiT-Tiny, a different configuration is used because DeiT-Tiny is a transformer-based architecture. Vision transformers usually rely more heavily on regularization and are often more sensitive to optimizer settings than convolutional neural networks. Therefore, AdamW is used instead of Adam. AdamW is selected because it decouples weight decay from the gradient update. This is important for transformer-based models because weight decay is commonly used as a regularization technique to control overfitting and improve generalization. In standard Adam, weight decay is coupled with the adaptive gradient update, which may make regularization less direct. AdamW handles this more appropriately by applying weight decay separately. For DeiT-Tiny, the learning rate is set to:

$$\eta = 0.0003.$$

This learning rate is smaller than the value used for the convolution-based models. The reason is that transformer models can be more sensitive to large parameter updates, especially when trained from scratch on small images such as 32×32 CIFAKE images. A smaller learning rate helps make training more stable and reduces the risk of unstable loss behavior. The weight decay for DeiT-Tiny is set to:

$$\lambda = 0.05.$$

This value is used to regularize the model parameters during training. Since DeiT-Tiny uses a transformer structure with self-attention layers, it can easily overfit if the dataset is not large enough or if the model learns overly specific patterns from the training set. Weight decay discourages excessively large weights and helps the model learn more generalizable representations. The batch size for DeiT-Tiny is set to:

$$B = 32.$$

This batch size is smaller than the batch size used for the convolution-based models. The main reason is that transformer-based models usually require more memory due to the self-attention mechanism. Reducing the batch size helps fit the model into memory during training. Although a smaller batch size can make gradient estimates noisier, it is a practical choice for balancing memory usage and training feasibility. The maximum number of epochs for DeiT-Tiny is set to:

$$E_{\max} = 30.$$

This is lower than the maximum epoch setting used for the convolution-based models. DeiT-Tiny is more expensive to train, and training for too many epochs may increase the risk of overfitting. Therefore, 30 epochs are used as a reasonable upper limit. The patience value for DeiT-Tiny is set to:

$$\text{patience} = 8.$$

This patience value is larger than the value used for the convolution-based models. The reason is that transformer models may have more unstable validation loss in the early stages of training, especially when trained from scratch. A larger patience value gives

DeiT-Tiny more time to improve before early stopping is triggered. This avoids stopping the model too early due to temporary fluctuations in validation performance. Therefore, the training configuration for DeiT-Tiny is:

optimizer = AdamW, $\eta = 0.0003$, $\lambda = 0.05$, $B = 32$, $E_{\max} = 30$, patience = 8.

Overall, the hyperparameter choices are designed to match the training behavior of each architecture group. The convolution-based models use Adam with a larger batch size and a standard learning rate because they are more stable and efficient on small images. DeiT-Tiny uses AdamW, a smaller learning rate, weight decay, and a smaller batch size because transformer-based models are more sensitive to optimization settings and usually require stronger regularization. Early stopping is applied to all models to select the best validation checkpoint and reduce overfitting.

4.8 Training Loop and Checkpoint Selection

Purpose: Explain how training is executed.

Main points:

- Each epoch uses `model.train()`.
- For each batch:
 - move images and labels to device,
 - reset gradients,
 - forward pass,
 - compute loss,
 - backward pass,
 - optimizer step,
 - compute batch predictions using sigmoid and threshold 0.5.
- Validation is performed after each epoch.
- Best checkpoint is selected by lowest validation loss.
- Early stopping is triggered when validation loss does not improve for the patience value.

4.9 Evaluation Protocol

After training, each model is evaluated using a fixed evaluation protocol. The purpose of this protocol is to measure the performance of the trained model without changing its parameters. In other words, the evaluation stage is only used for inference and metric computation, not for learning.

During validation and testing, the model is first switched to evaluation mode by calling:

`model.eval()`.

This step is important because some layers behave differently during training and evaluation. For example, dropout layers are disabled during evaluation, and batch normalization layers use stored statistics instead of updating their running statistics. Therefore, using `model.eval()` ensures that the model produces deterministic and stable predictions during validation and testing.

The evaluation loop is also executed inside:

```
torch.no_grad().
```

This tells PyTorch not to track gradients during the forward pass. Since evaluation does not perform backpropagation, gradients are not needed. Disabling gradient computation reduces memory usage and makes evaluation faster. More importantly, it also makes clear that the model parameters are not updated during validation or testing.

For each model, the best checkpoint saved during training is loaded before final testing. The best checkpoint is selected based on validation performance, usually the lowest validation loss. This means that the final test result is not computed from the last epoch automatically, but from the model state that performed best on the validation set.

Let θ^* denote the parameters loaded from the best checkpoint. During testing, the model computes logits for the test images:

$$z_i = f_{\theta^*}(x_i),$$

where x_i is the i -th test image and z_i is the corresponding output logit. The logit is then converted into a probability using the sigmoid function:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

The predicted label is obtained using a threshold of 0.5:

$$\hat{y}_i = \begin{cases} 1, & \text{if } p_i \geq 0.5, \\ 0, & \text{if } p_i < 0.5. \end{cases}$$

Here, label 1 corresponds to **REAL**, and label 0 corresponds to **FAKE**. Therefore, if the predicted probability is greater than or equal to 0.5, the image is classified as **REAL**; otherwise, it is classified as **FAKE**.

The evaluation loop does not call `optimizer.zero_grad()`, `loss.backward()`, or `optimizer.step()`. Therefore, no gradient is computed and no model weight is updated. The loop only performs the following operations:

input image $\rightarrow$ forward pass $\rightarrow$ logit $\rightarrow$ probability $\rightarrow$ predicted label $\rightarrow$ loss and metric computation.

The validation set is used during training to monitor generalization and select the best checkpoint. The test set is used only after training is finished. This separation is important because the test set should represent unseen data and should not influence model selection during training.

After evaluation, the computed results are saved into a JSON file. This makes the experimental results easier to store, compare, and reproduce. For each model, the saved JSON file contains values such as average loss, accuracy, precision, recall, F1-score, and the confusion-matrix components. This allows the performance of different models to be compared using the same evaluation criteria.

4.10 Evaluation Metrics

The performance of each model is measured using average loss, accuracy, precision, recall, and F1-score. These metrics are computed from the predicted labels and the ground-truth labels on the validation or test set.

For binary classification, each prediction belongs to one of four cases: true positive, true negative, false positive, or false negative. In this project, the positive class is **REAL** and the negative class is **FAKE**.

$$\text{Positive class} = \text{REAL} = 1, \quad \text{Negative class} = \text{FAKE} = 0.$$

The four confusion-matrix components are defined as follows:

$$\begin{aligned} TP &= \sum_{i=1}^N \mathbb{I}(y_i = 1 \wedge \hat{y}_i = 1), \\ TN &= \sum_{i=1}^N \mathbb{I}(y_i = 0 \wedge \hat{y}_i = 0), \\ FP &= \sum_{i=1}^N \mathbb{I}(y_i = 0 \wedge \hat{y}_i = 1), \\ FN &= \sum_{i=1}^N \mathbb{I}(y_i = 1 \wedge \hat{y}_i = 0). \end{aligned}$$

Here, N is the total number of evaluated samples, y_i is the true label, $\hat{y}_i$ is the predicted label, and $\mathbb{I}(\cdot)$ is the indicator function. The indicator function returns 1 if the condition is true and 0 otherwise.

The meaning of each component is:

- TP means that the image is actually **REAL** and the model also predicts **REAL**.
- TN means that the image is actually **FAKE** and the model also predicts **FAKE**.
- FP means that the image is actually **FAKE**, but the model incorrectly predicts **REAL**.
- FN means that the image is actually **REAL**, but the model incorrectly predicts **FAKE**.

This can be summarized using the following confusion matrix:

	Predicted REAL	Predicted FAKE
Actual REAL	TP	FN
Actual FAKE	FP	TN

The average evaluation loss is computed by averaging the loss over all evaluated samples. If $\mathcal{L}_i$ is the binary cross-entropy loss of the i -th sample, the average loss is:

$$\mathcal{L}_{avg} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i.$$

In practice, the loss may be computed batch by batch. In that case, the total loss should be accumulated according to the number of samples in each batch, then divided

by the total number of samples. This avoids giving the last smaller batch the same weight as a full batch.

Accuracy measures the proportion of correctly classified samples among all samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Accuracy is easy to understand because it shows the overall percentage of correct predictions. However, accuracy alone may not be enough if the model makes different types of mistakes. Therefore, precision, recall, and F1-score are also reported.

Precision measures how many images predicted as **REAL** are actually **REAL**:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

A high precision means that when the model predicts an image as **REAL**, that prediction is usually correct. In this project, low precision means the model produces many false positives, meaning many **FAKE** images are incorrectly classified as **REAL**.

Recall measures how many actual **REAL** images are correctly detected by the model:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

A high recall means that the model can find most of the **REAL** images in the dataset. Low recall means that many **REAL** images are missed and incorrectly classified as **FAKE**.

F1-score is the harmonic mean of precision and recall:

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Equivalently, F1-score can also be written directly using TP , FP , and FN :

$$\text{F1-score} = \frac{2TP}{2TP + FP + FN}.$$

F1-score is useful because it balances precision and recall. A model can only obtain a high F1-score when both precision and recall are high. Therefore, F1-score is more informative than accuracy when we want to evaluate whether the model performs well on the positive class, **REAL**, without ignoring false positives and false negatives.

In the implementation, division by zero is handled safely. For example, if $TP + FP = 0$, precision is defined as 0 instead of causing a runtime error. Similarly, if $TP + FN = 0$, recall is defined as 0. This makes the metric computation stable even in rare cases where the model predicts only one class.

Overall, the evaluation metrics provide complementary views of model performance. Average loss measures the quality of the predicted probabilities. Accuracy measures the overall correctness. Precision measures the reliability of **REAL** predictions. Recall measures the ability to detect actual **REAL** images. F1-score combines precision and recall into a single balanced metric.

Chapter 5

Training Behavior Analysis

5.1 Training History and Analysis Method

This chapter analyzes how each model behaves during training. Instead of looking only at the final test result, the training history is used to observe how the model learns from epoch to epoch. For each epoch, the recorded metrics include loss, accuracy, precision, recall, and F1-score on both the training set and the validation set.

The main purpose of this chapter is to check whether each model learns stably and whether it shows signs of overfitting or underfitting. Training loss shows how well the model fits the training data, while validation loss shows how well the model generalizes to unseen validation data. If training loss keeps decreasing but validation loss stops improving or starts increasing, the model may be overfitting.

Besides loss, accuracy and F1-score are also analyzed. Accuracy shows the overall classification correctness, while F1-score provides a more balanced view between precision and recall. These metrics are useful because the task is binary REAL/FAKE classification, and the model should perform well on both classes rather than only improving one type of prediction.

The best epoch is selected based on the lowest validation loss. Therefore, the best checkpoint is not always the final epoch. This is important because the final epoch may already be affected by overfitting. In this chapter, the test set is not used for training behavior analysis. The test set is reserved for the final model comparison in the next chapter.

Overall, this chapter focuses on training and validation curves, best validation epoch, early stopping behavior, and the train-validation gap. These observations help explain not only which model performs well, but also how each model reaches its performance during training.

5.2 SimpleCNN Training Behavior

Figure 5.1 shows the training behavior of the SimpleCNN model across epochs. The figure includes three curves: loss, accuracy, and F1-score. Each subplot compares the training curve with the validation curve, and the dashed vertical line marks the best epoch selected by the lowest validation loss.

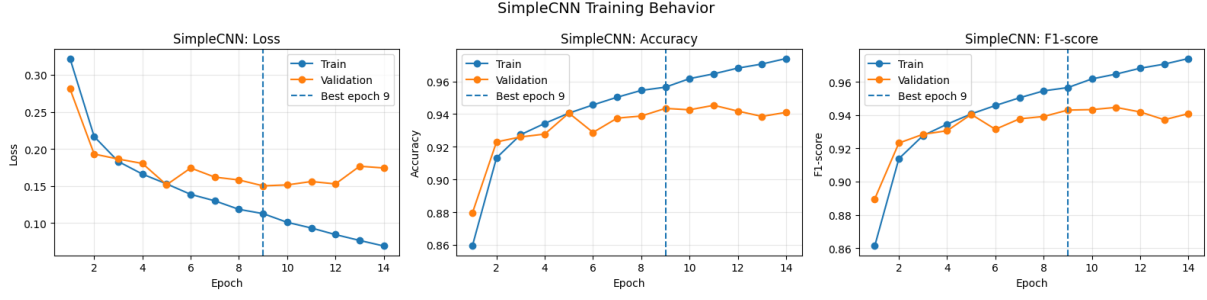


Figure 5.1: Training and validation behavior of SimpleCNN across epochs.

The loss curve shows that SimpleCNN learns quickly during the first few epochs. Both training loss and validation loss decrease strongly at the beginning, which indicates that the model can capture useful REAL/FAKE visual cues from the CIFAKE dataset. After the early epochs, the training loss continues to decrease steadily, while the validation loss becomes less stable.

The best checkpoint is selected at epoch 9 because this epoch has the lowest validation loss. After epoch 9, the training loss continues to decrease, but the validation loss does not improve further and starts to fluctuate. This behavior suggests that the model begins to fit the training data more strongly after the best epoch, while its generalization to validation images does not improve at the same rate.

The accuracy and F1-score curves show a similar pattern. Training accuracy and training F1-score continue to increase until the final epoch. In contrast, validation accuracy and validation F1-score improve quickly at first but then become relatively stable. This creates a moderate train-validation gap near the later epochs.

Overall, SimpleCNN shows stable learning behavior and reaches a strong validation performance despite its simple architecture. However, the gap between the continuously improving training curves and the more stable validation curves indicates mild overfitting after the best validation epoch. This is expected because SimpleCNN has enough capacity to fit low-level patterns in the training data, but its simpler architecture may limit its ability to generalize further compared with deeper CNN models.

5.3 ResNet18 Training Behavior

Figure 5.2 shows the training behavior of ResNet18 across epochs. The figure presents the training and validation curves for loss, accuracy, and F1-score. The dashed vertical line indicates the best epoch selected by the lowest validation loss.

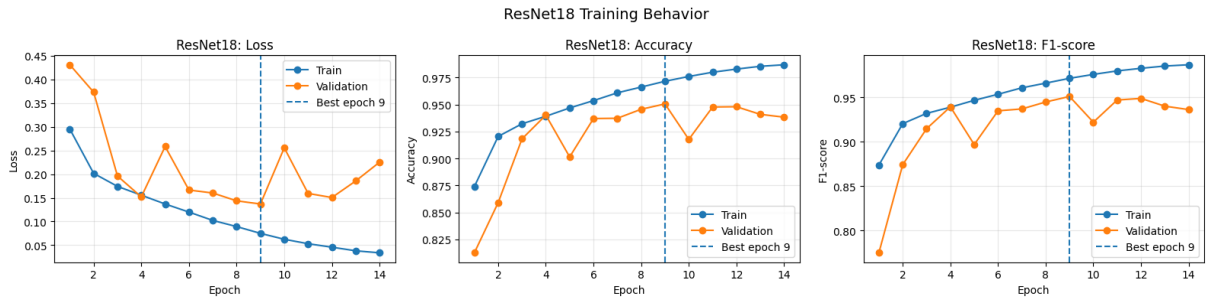


Figure 5.2: Training and validation behavior of ResNet18 across epochs.

The loss curve shows that ResNet18 learns effectively during the early training stage. Both training loss and validation loss decrease sharply in the first few epochs, which indicates that the model quickly captures useful discriminative patterns for REAL/FAKE classification. Compared with SimpleCNN, ResNet18 reaches a lower training loss and continues improving more strongly on the training set.

The best checkpoint is selected at epoch 9 because this epoch gives the lowest validation loss. After epoch 9, the training loss continues to decrease steadily, while the validation loss becomes more unstable and increases noticeably in several later epochs. This behavior suggests that the model continues fitting the training data after the optimal validation point, but this additional fitting does not consistently improve generalization.

The accuracy and F1-score curves support the same observation. Training accuracy and training F1-score continue to improve almost monotonically until the end of training, reaching very high values. In contrast, validation accuracy and validation F1-score improve overall but fluctuate more strongly. Although the validation performance remains relatively high, it does not improve as smoothly as the training performance, and several drops can be observed after the best epoch.

This pattern indicates that ResNet18 has stronger representation capacity than SimpleCNN, which allows it to fit the training data more effectively. The residual learning structure helps optimization and supports stable convergence. However, the increasing gap between training and validation curves after the best epoch suggests mild to moderate overfitting.

Overall, ResNet18 demonstrates strong learning ability and achieves high validation performance, but its later training behavior shows that better fitting on the training set does not always translate into better validation generalization. This makes the best validation epoch an important point for checkpoint selection.

5.4 EfficientNet-B0 Training Behavior

Figure 5.3 shows the training behavior of EfficientNet-B0 across epochs. The figure presents the training and validation curves for loss, accuracy, and F1-score. The dashed vertical line marks the best epoch selected by the lowest validation loss.



Figure 5.3: Training and validation behavior of EfficientNet-B0 across epochs.

The loss curve shows that EfficientNet-B0 learns quickly during the first few epochs. Both training loss and validation loss decrease substantially at the beginning, which indicates that the model is able to extract useful discriminative features for REAL/FAKE classification. After this early learning stage, the training loss continues to decrease steadily, while the validation loss becomes much more stable and remains within a relatively narrow range.

The best checkpoint is selected at epoch 9 because this epoch achieves the lowest validation loss. After epoch 9, the training loss continues to decrease, but the validation loss no longer improves meaningfully and stays nearly flat. This suggests that the model continues fitting the training data, while its generalization performance on the validation set has already reached a plateau.

The accuracy and F1-score curves support the same interpretation. Training accuracy and training F1-score continue to increase gradually until the last epoch. In contrast, validation accuracy and validation F1-score rise quickly in the early epochs and then remain highly stable. The fluctuations in the validation curves are relatively small compared with those of some other models, which indicates that EfficientNet-B0 has a stable validation behavior.

Compared with ResNet18, EfficientNet-B0 appears to generalize more smoothly because its validation performance does not fluctuate as strongly after the best epoch. Although a train-validation gap still exists, especially in the later epochs, the gap grows in a more controlled manner. This suggests that EfficientNet-B0 benefits from its efficient architecture design and is able to maintain a good balance between fitting the training data and preserving validation performance.

Overall, EfficientNet-B0 demonstrates stable learning behavior and strong validation consistency. The model shows mild overfitting after the best validation epoch, but the effect is not severe. This makes EfficientNet-B0 a well-balanced CNN model in terms of both optimization stability and generalization behavior.

5.5 DenseNet121 Training Behavior

Figure 5.4 shows the training behavior of DenseNet121 across epochs. The figure presents the training and validation curves for loss, accuracy, and F1-score. The dashed vertical line marks the best epoch selected by the lowest validation loss.

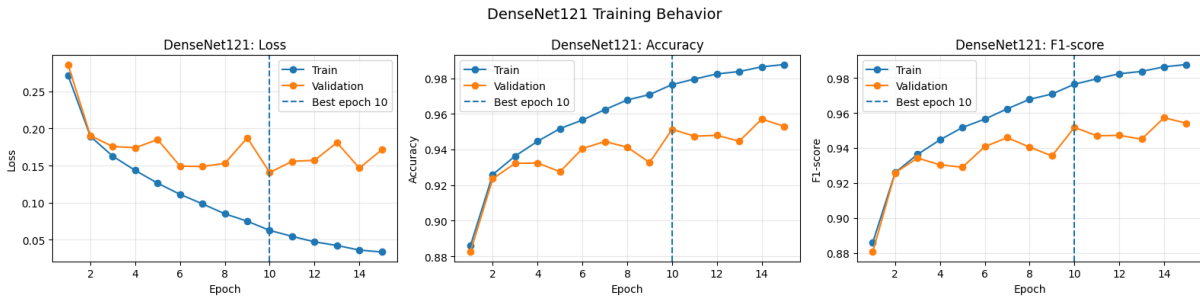


Figure 5.4: Training and validation behavior of DenseNet121 across epochs.

The loss curve shows that DenseNet121 learns effectively from the beginning of training. Both training loss and validation loss decrease clearly during the early epochs, which indicates that the model quickly captures useful discriminative features for REAL/FAKE classification. Compared with the previous CNN models, DenseNet121 achieves a very low training loss and continues reducing it steadily throughout training.

The best checkpoint is selected at epoch 10 because this epoch gives the lowest validation loss. After epoch 10, the training loss continues to decrease strongly, while the validation loss no longer improves and instead fluctuates across later epochs. This behavior indicates that the model continues fitting the training data after the best validation point, but the additional fitting does not consistently improve validation generalization.

The accuracy and F1-score curves show the same pattern. Training accuracy and training F1-score increase almost monotonically and reach very high values by the end of training. In contrast, validation accuracy and validation F1-score improve overall but remain less stable than the training curves. Although the validation performance is strong, several fluctuations can be observed after the best epoch, especially in the later training stage.

This pattern suggests that DenseNet121 has a strong representation capacity and is highly effective at learning the visual patterns needed for the CIFAKE task. The dense connectivity structure likely helps the model reuse low-level and mid-level features, which is useful for detecting subtle synthetic-image artifacts. However, the steadily improving training curves together with the less stable validation curves also indicate mild overfitting after the best validation epoch.

Overall, DenseNet121 demonstrates strong learning ability and high validation performance. Among the CNN-based models, it shows particularly powerful fitting capacity and reaches a strong validation level, while still maintaining relatively good generalization. The best validation checkpoint at epoch 10 is therefore an important reference point for final evaluation.

5.6 DeiT-Tiny Training Behavior

Figure 5.5 shows the training behavior of DeiT-Tiny across epochs. The figure presents the training and validation curves for loss, accuracy, and F1-score. The dashed vertical line marks the best epoch selected by the lowest validation loss.

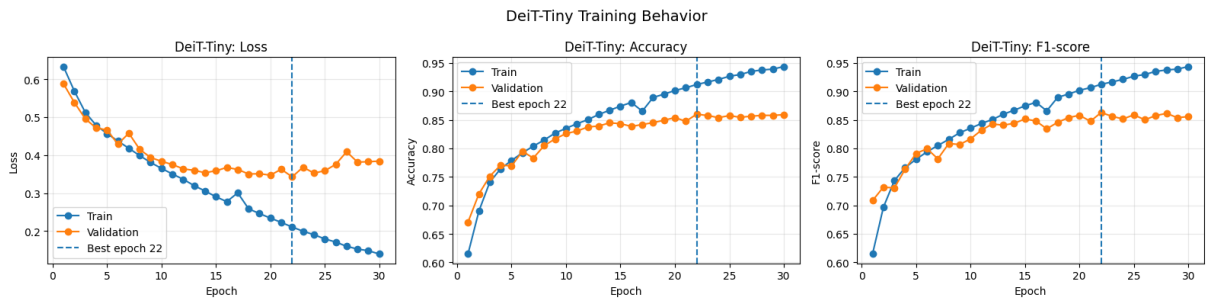


Figure 5.5: Training and validation behavior of DeiT-Tiny across epochs.

The loss curve shows that DeiT-Tiny learns more gradually than the CNN-based models. Both training loss and validation loss decrease clearly during the early epochs, which indicates that the model is able to learn useful patterns for REAL/FAKE classification. However, the overall loss values remain higher than those of the CNN models, and the decrease is slower, which suggests that optimization is more difficult for this Transformer-based architecture under the current training setup.

The best checkpoint is selected at epoch 22 because this epoch gives the lowest validation loss. Before this point, the validation loss decreases overall but with noticeable fluctuations. After epoch 22, the training loss continues to decrease, while the validation loss becomes unstable and tends to remain flat or increase slightly. This behavior indicates that the model continues fitting the training data, but the improvement on validation generalization becomes limited after the best epoch.

The accuracy and F1-score curves show a similar pattern. Training accuracy and training F1-score continue to improve steadily until the end of training. In contrast,

validation accuracy and validation F1-score improve during the early and middle epochs, but their growth becomes much slower later on. After the best epoch, the validation curves remain relatively stable but do not follow the same upward trend as the training curves. As a result, the train-validation gap becomes more visible in the later training stage.

Compared with the CNN-based models, DeiT-Tiny shows a larger gap between training and validation performance. This suggests that, under the current setup, the model has weaker generalization than the best CNN models. One possible reason is that Transformer-based image models usually rely less on built-in local spatial inductive bias than CNNs. For a small-resolution dataset such as CIFAKE, this may make optimization harder, especially when the model is trained from scratch without pretrained weights and without stronger data augmentation.

Overall, DeiT-Tiny is able to learn meaningful discriminative features and reaches a reasonable validation performance, but its training behavior is less stable and less efficient than that of the CNN-based models. The model shows a clearer train-validation gap and more limited validation improvement in the later epochs, which suggests that the current setup is not fully optimal for this architecture.

5.7 Cross-Model Training Comparison

Figure 5.6 compares the validation loss curves of all five models. The CNN-based models, including SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121, reach low validation loss within the first few epochs. This shows that CNN architectures are able to learn useful REAL/FAKE visual cues quickly on the CIFAKE dataset.



Figure 5.6: Validation loss comparison across all models.

Among the CNN models, EfficientNet-B0 and DenseNet121 show particularly strong validation loss behavior. Their validation loss remains low and relatively stable after the early epochs. SimpleCNN also reaches a competitive validation loss, but its curve

shows that a simple architecture may have less capacity to improve further after the early training stage. ResNet18 learns quickly, but its validation loss fluctuates more strongly in later epochs, suggesting less stable validation behavior compared with EfficientNet-B0 and DenseNet121.

DeiT-Tiny behaves differently from the CNN-based models. Its validation loss starts much higher and decreases more slowly. Even after many epochs, its validation loss remains clearly above the CNN models. This suggests that DeiT-Tiny is harder to optimize under the current setup. Since CIFAKE images are only 32×32 pixels, CNN models may benefit more from their local spatial inductive bias, while the Transformer-based DeiT-Tiny may require stronger training strategies such as pretrained weights, data augmentation, or learning rate scheduling.

Figure 5.7 compares the validation F1-score curves of all models. The CNN-based models quickly reach high validation F1-scores, while DeiT-Tiny improves more gradually and stays at a lower level.

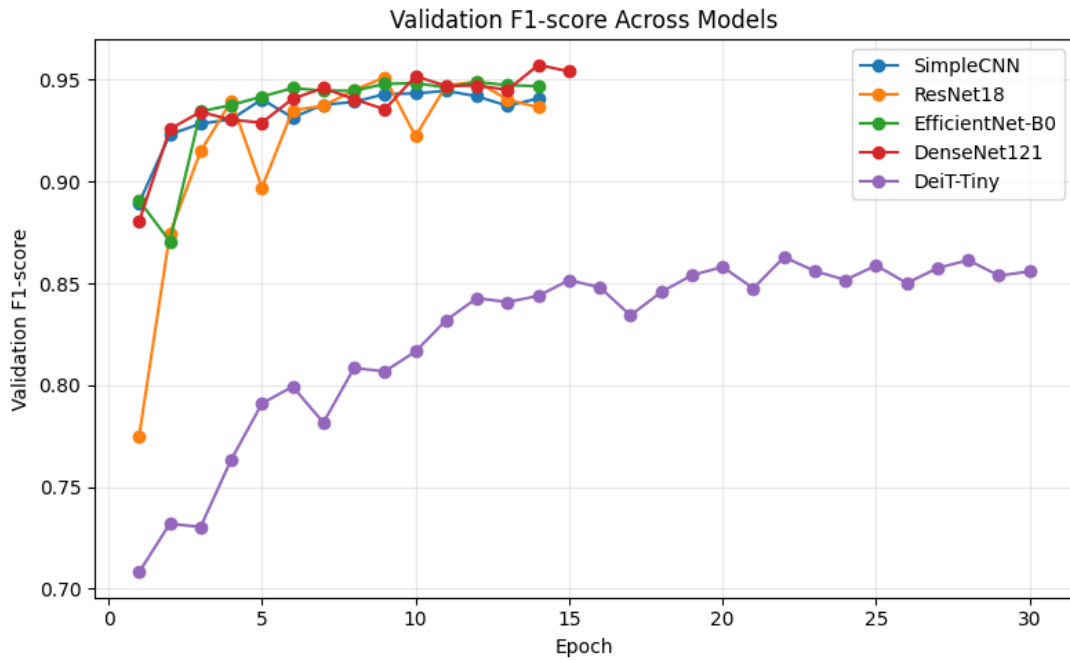


Figure 5.7: Validation F1-score comparison across all models.

The validation F1-score comparison confirms the trend observed in the validation loss curves. CNN-based models reach strong validation performance early and remain close to each other, while DeiT-Tiny shows a much slower learning process. DenseNet121 achieves one of the strongest validation F1-score levels, which supports the idea that dense feature reuse is helpful for detecting subtle real-versus-synthetic cues. EfficientNet-B0 also shows stable validation F1-score behavior, indicating a good balance between learning capacity and generalization.

Overall, the cross-model training comparison shows that CNN-based models are more suitable for the current CIFAKE setup than DeiT-Tiny. The CNN models learn faster, reach lower validation loss, and achieve higher validation F1-scores. DeiT-Tiny still learns meaningful patterns, but its validation performance is lower and improves more slowly. This suggests that local texture, edge, and artifact-level features are especially important for CIFAKE classification, and CNN architectures are better aligned with these visual characteristics under the current training configuration.

Chapter 6

Experimental Results and Discussion

6.1 Final Test Results

After completing training and selecting the best checkpoint for each model based on validation loss, the final evaluation was performed on the test set. The goal of this chapter is to compare the five models on unseen data and analyze their relative strengths and weaknesses. The evaluated models are SimpleCNN, ResNet18, EfficientNet-B0, DenseNet121, and DeiT-Tiny.

This table summarizes the final test performance of all models. The reported metrics include test loss, accuracy, precision, recall, and F1-score. Since this is a binary REAL/FAKE classification problem, these metrics provide a more complete evaluation than accuracy alone.

rank	model	best_epoch	test_loss	test_accuracy	test_precision	test_recall	test_f1
1	DenseNet121	10	0.136493	0.95120	0.944007	0.9593	0.951592
2	EfficientNet-B0	9	0.129921	0.94955	0.950045	0.9490	0.949522
3	ResNet18	9	0.137099	0.94800	0.941902	0.9549	0.948356
4	SimpleCNN	9	0.150262	0.94265	0.953117	0.9311	0.941980
5	DeiT-Tiny	22	0.336656	0.86115	0.848701	0.8790	0.863585

Figure 6.1: Final test performance comparison across models.

Figure 6.2 provides a visual comparison of the final test metrics across all models. The figure shows that the CNN-based models achieve consistently strong performance, while DeiT-Tiny performs noticeably worse under the current training setup.

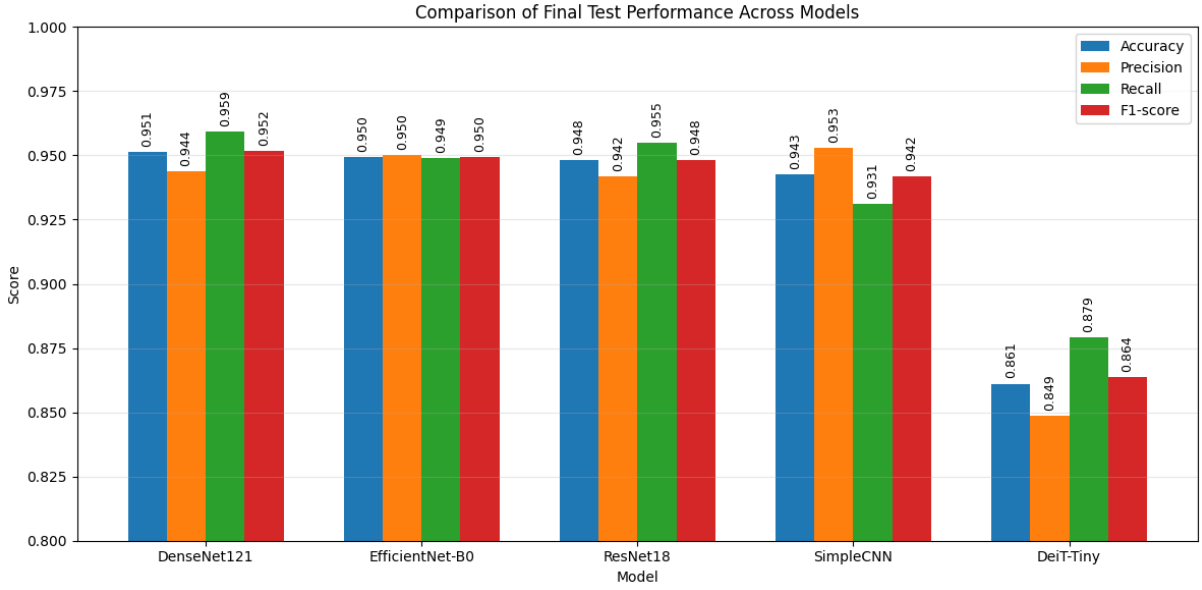


Figure 6.2: Comparison of final test performance across models.

The results show that DenseNet121 achieves the best overall performance, with the highest F1-score of 0.9516 and the highest accuracy of 0.9512. This suggests that DenseNet121 provides the best balance between precision and recall on the CIFAKE test set. Its strong performance is consistent with the training behavior analysis in the previous chapter, where DenseNet121 showed both effective feature learning and strong validation performance.

EfficientNet-B0 ranks second overall. Although its F1-score is slightly lower than DenseNet121, it achieves the lowest test loss among all models. This indicates that EfficientNet-B0 produces very confident and well-calibrated predictions. Its precision, recall, and accuracy are also highly competitive, which confirms that it is one of the strongest and most stable models in this study.

ResNet18 also performs very well and ranks third. Its results are close to those of DenseNet121 and EfficientNet-B0, but its test metrics are slightly lower overall. The model still demonstrates strong generalization ability, showing that residual learning is effective for the CIFAKE classification task.

SimpleCNN, despite being the simplest architecture in the experiment, achieves a surprisingly competitive result. Its F1-score of 0.9420 is only slightly below the deeper CNN models. This suggests that even a relatively simple convolutional network can learn useful low-level features for distinguishing REAL and FAKE images. However, its lower recall and lower overall ranking indicate that its representational capacity is still more limited than that of the deeper architectures.

DeiT-Tiny shows the weakest final performance. Its loss is much higher, and its accuracy, precision, recall, and F1-score are all clearly below the CNN-based models. This result is consistent with the training behavior analysis, where DeiT-Tiny showed slower convergence, higher validation loss, and a larger train-validation gap. One possible explanation is that Transformer-based models typically require stronger optimization strategies, larger-scale pretraining, or richer augmentation pipelines. In this project, DeiT-Tiny was trained from scratch on small 32×32 images, which may not be an ideal setting for this architecture.

Overall, the final test results indicate that CNN-based models are more suitable than

DeiT-Tiny for the current CIFAKE setup. Among them, DenseNet121 achieves the best overall trade-off between accuracy, precision, recall, and F1-score, while EfficientNet-B0 also provides highly competitive performance with the lowest test loss. These results suggest that dense connectivity and efficient convolutional feature extraction are especially effective for detecting subtle synthetic-image artifacts in low-resolution images.

6.2 Discussion of Model Performance

The performance differences across models can be explained by both architecture design and dataset characteristics. CIFAKE is a binary image classification dataset in which the distinction between classes depends less on semantic object identity and more on subtle texture inconsistencies, edge artifacts, background irregularities, and synthetic visual patterns. For this reason, local feature extraction is especially important.

CNN-based models are naturally well suited to this task because convolutional filters are effective at capturing local spatial patterns. DenseNet121 benefits from dense feature reuse, which helps preserve and combine low-level and mid-level visual cues across layers. EfficientNet-B0 benefits from a balanced architecture design that achieves strong performance with good efficiency. ResNet18 benefits from stable optimization through residual connections. Even SimpleCNN performs well because the task still relies strongly on local visual evidence.

By contrast, DeiT-Tiny relies on a Transformer-based patch representation. Without pretrained weights or stronger augmentation, this architecture may struggle more on a small-resolution dataset such as CIFAKE. This may explain why its performance remains substantially below the CNN models in the current experiment.

In summary, the experimental results support the conclusion that CNN architectures are more effective than DeiT-Tiny for this project setting, and DenseNet121 is the best-performing model overall.

Chapter 7

Explainability Analysis with Grad-CAM

7.1 Purpose and Scope of Explainability Analysis

The previous chapters evaluate model performance using training curves, validation behavior, and final test metrics. However, high accuracy or high F1-score alone does not fully explain how a model makes its decision. A model may classify an image correctly, but it may rely on background patterns, texture bias, or generator-specific shortcuts instead of meaningful visual evidence. Therefore, explainability analysis is included to inspect which image regions contribute most strongly to model predictions.

In this project, Grad-CAM is used as the main explainability method. However, the Grad-CAM analysis is applied only to the CNN-based models: SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121. These models contain convolutional feature maps, which are suitable for Grad-CAM visualization. Their heatmaps can therefore be compared in terms of activated regions, coverage, entropy, and peak value.

DeiT-Tiny is not included in the Grad-CAM comparison because it is a Transformer-based model. Unlike CNNs, DeiT-Tiny processes the image as a sequence of patches and uses self-attention rather than convolutional feature maps in the same way as CNN architectures. As a result, standard CNN-style Grad-CAM heatmaps are not directly comparable with DeiT-Tiny attention behavior. For DeiT-Tiny, Transformer-specific explanation methods such as attention visualization or attention rollout would be more appropriate and are left as future work.

The goal of this chapter is therefore not to compare all five models using Grad-CAM, but to analyze how the four CNN-based models make predictions on REAL and FAKE images. Since CIFAKE classification is not ordinary object recognition, the analysis does not only check whether the model focuses on the main object. It also examines whether the model attends to texture, edges, background regions, or artifact-like patterns.

7.2 Grad-CAM Method

Grad-CAM, or Gradient-weighted Class Activation Mapping, generates a heatmap by using the gradients of a target class score with respect to feature maps in a convolutional layer. Let A^k denote the k -th feature map of a convolutional layer, and let y^c denote the score for class c . Grad-CAM first computes the importance weight α_k^c for each feature map.

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \quad (7.1)$$

where Z is the number of spatial locations in the feature map. These weights represent how important each feature map is for the target class.

The Grad-CAM heatmap is then computed as:

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right). \quad (7.2)$$

The ReLU operation keeps only positive contributions to the target class. The resulting heatmap is resized to the original image size and overlaid on the input image. Bright or red regions usually indicate areas that contribute more strongly to the model prediction.

For the CIFAKE task, this visualization is especially useful because the label depends on whether the image is real or AI-generated, not on the object category itself. Therefore, important evidence may appear not only on the main object, but also in background regions, edges, textures, or local artifacts.

7.3 Qualitative Grad-CAM Results

Grad-CAM visualizations are used to compare how the four CNN-based models focus on image regions. The analysis includes REAL images, FAKE images, correct predictions, and incorrect predictions. This helps determine whether the models rely on object-level regions, background regions, textures, or artifact-like areas.

Figure 7.1 shows Grad-CAM visualizations on a REAL image.

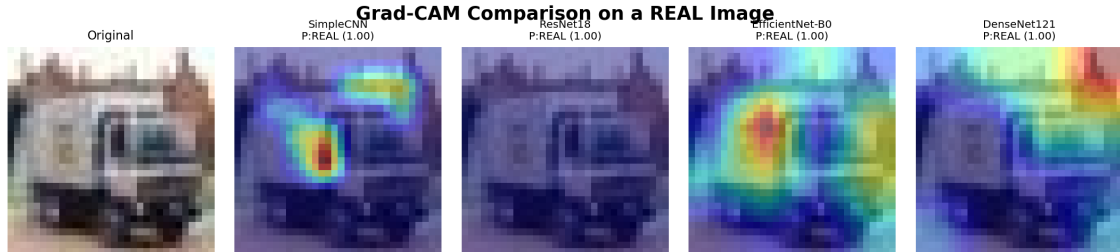


Figure 7.1: Grad-CAM comparison of CNN-based models on a REAL image.

For REAL images, the heatmaps show which regions the models consider important when predicting the image as real. Ideally, the model should focus on meaningful visual structures such as object boundaries, texture regions, or visually informative areas. However, because CIFAKE classification is not ordinary object recognition, attention may also appear on background or local texture regions. This is reasonable because real-versus-synthetic cues can exist outside the main object.

Figure 7.2 shows Grad-CAM visualizations on a FAKE image.



Figure 7.2: Grad-CAM comparison of CNN-based models on a FAKE image.

For FAKE images, the heatmaps help inspect whether the models focus on possible synthetic artifacts. These artifacts may appear as unnatural textures, inconsistent edges, unusual background patterns, or local distortions. The visual results suggest that the models do not rely only on semantic object-level information. Instead, they may use texture, background, edge, and artifact-like regions as evidence for REAL/FAKE classification.

Figure 7.3 shows examples of incorrect predictions and their Grad-CAM explanations.

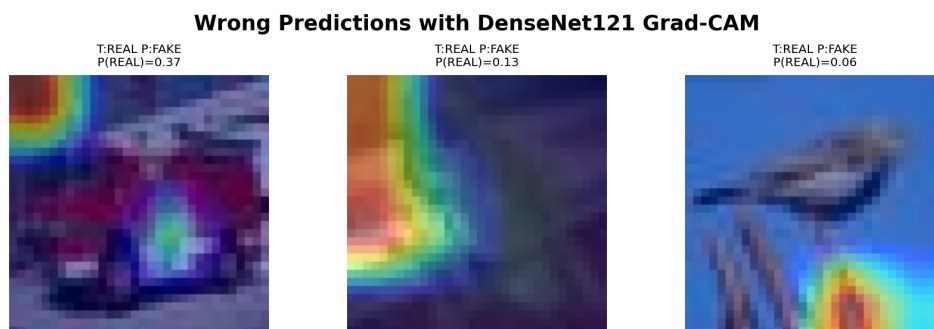


Figure 7.3: Examples of incorrect predictions and their Grad-CAM explanations.

Incorrect predictions are especially useful for understanding model limitations. If a model focuses on irrelevant background regions or visually weak areas, the prediction may become unreliable. These cases show that a high test score does not guarantee perfect reasoning. The model may still depend on shortcut cues or dataset-specific patterns.

7.4 Quantitative Heatmap Analysis

Besides visual inspection, heatmaps are also analyzed using simple quantitative statistics. These statistics help compare the spatial behavior of different CNN models more objectively.

The first statistic is **coverage**, which measures the proportion of pixels with high heatmap activation. A low coverage value means the heatmap is concentrated in a small region, while a high coverage value means the activation is more spread out.

The second statistic is **normalized entropy**, which measures how distributed the heatmap activation is. Higher entropy means the model attention is more spread across the image, while lower entropy means the attention is more concentrated.

The third statistic is **peak value**, which represents the maximum activation strength after normalization. This value indicates whether the heatmap contains a strong focused activation.

Table 7.1 summarizes the average normalized heatmap statistics for the four CNN-based models.

Table 7.1: Average normalized Grad-CAM heatmap statistics for CNN-based models

Model	Coverage	Normalized Entropy	Peak Value
SimpleCNN	0.052	0.701	0.93
ResNet18	0.061	0.534	0.66
EfficientNet-B0	0.277	0.926	1.00
DenseNet121	0.203	0.817	0.94

The statistics show that better classification performance does not always mean the most spatially concentrated attention. ResNet18 has relatively low coverage and entropy, which suggests more concentrated activation. EfficientNet-B0 has the highest coverage and entropy, meaning its heatmaps are more spatially distributed. DenseNet121, which achieves the best final classification result, has broader attention than ResNet18 but still maintains strong peak activation.

This indicates that different CNN architectures may use different spatial evidence for the same REAL/FAKE task. A model with broader attention may combine information from multiple regions, while a model with concentrated attention may rely on a smaller set of highly activated areas.

Figure 7.4 shows the distribution of Grad-CAM coverage values for each CNN-based model.

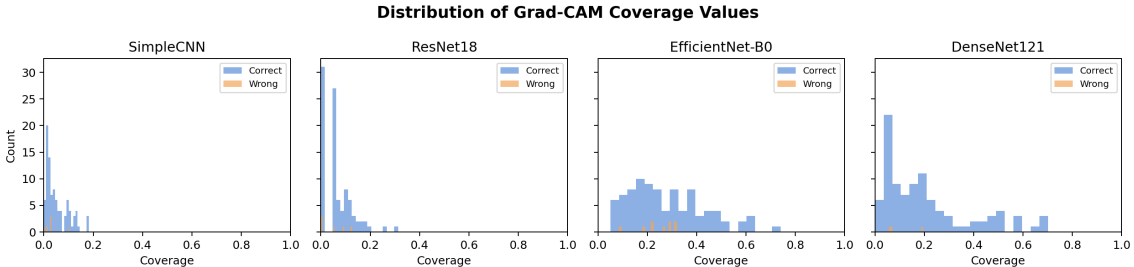


Figure 7.4: Distribution of Grad-CAM coverage values for CNN-based models.

The coverage distribution further supports the idea that the models do not attend to images in the same way. This is important because explainability analysis provides a different perspective from accuracy and F1-score. It helps reveal how models make predictions, not only whether those predictions are correct.

7.5 Discussion of Attention Patterns

The Grad-CAM analysis suggests that CIFAKE classification depends on more than object semantics. In ordinary object classification, it is often expected that the model focuses mainly on the object itself. In CIFAKE, however, the label depends on image origin, not object category. Therefore, useful evidence may appear in textures, edges, backgrounds, and local artifacts.

DenseNet121 achieves the best final test performance, but its attention is not necessarily the most concentrated. This suggests that strong performance may come from

combining information across multiple regions rather than focusing on only one small area. EfficientNet-B0 also shows broad heatmap behavior, which may reflect its ability to aggregate distributed visual evidence.

Overall, Grad-CAM supports the main interpretation of this project: REAL/FAKE detection is different from ordinary image classification. The CNN-based models may rely on subtle visual traces of the image generation process, and these traces can appear across different image regions.

Chapter 8

Gradio Demo

After training and evaluating the models, a Gradio demo is implemented to make the project easier to test and present. Instead of running prediction scripts manually, the user can upload an image through a simple web interface and receive the classification result directly. The demo is designed as an interactive application for REAL/FAKE image prediction and visual explanation.

In this project, the demo uses the best-performing CNN model selected from the experimental results. The uploaded image is first preprocessed in the same way as the test images. The trained model then predicts whether the image is REAL or FAKE. In addition to the predicted label, the demo also displays the prediction probabilities and a Grad-CAM visualization.

The demo provides the following outputs:

- predicted label: REAL or FAKE,
- probability of the image being REAL,
- probability of the image being FAKE,
- Grad-CAM heatmap,
- Grad-CAM overlay on the input image.

Figure [8.1](#) shows the Gradio demo interface.

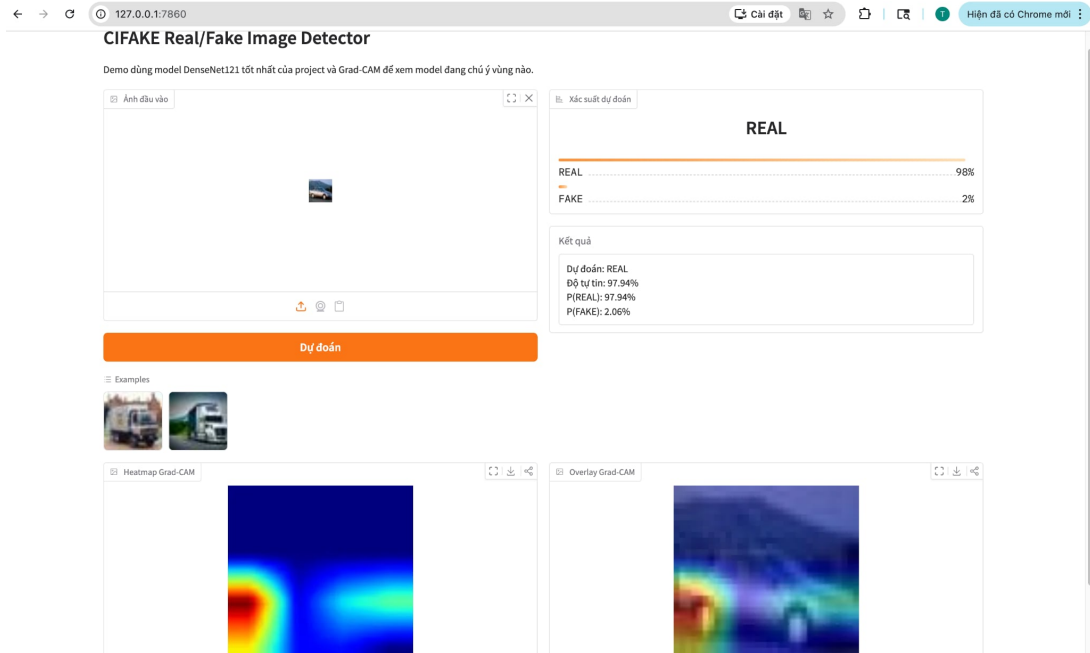


Figure 8.1: Gradio demo for REAL/FAKE prediction and Grad-CAM visualization.

The Gradio demo helps demonstrate the practical use of the project. It allows users to test individual images and immediately inspect both the classification output and the explanation heatmap. This is useful because explainability is an important part of the project. A prediction result alone only shows the final label, while the Grad-CAM overlay provides additional information about which image regions influence the prediction.

The demo also makes the project easier to present. During demonstration, the user can upload a CIFAKE image or another image, run the model, and visually inspect whether the prediction and Grad-CAM explanation are reasonable. Therefore, the demo serves not only as a testing interface, but also as a practical illustration of explainable AI for synthetic image detection.

Chapter 9

Limitations

Although the project successfully builds and evaluates a complete REAL/FAKE image classification pipeline, it still has several limitations.

First, CIFAKE images have a very low resolution of 32×32 pixels. This makes the classification task challenging because fine-grained visual details are limited. It also affects explainability analysis because Grad-CAM heatmaps become coarse when applied to low-resolution images.

Second, the dataset provides only binary labels: REAL and FAKE. This means the model cannot distinguish between different types of synthetic images, different generator architectures, prompt styles, or generation settings. As a result, the analysis cannot determine whether the classifier learns general fake-image features or generator-specific traces.

Third, the FAKE images in CIFAKE are generated from a specific synthetic-image generation process. Strong performance on CIFAKE does not necessarily mean that the model will generalize well to images generated by newer or different generative models. This is an important limitation for real-world fake-image detection.

Fourth, all models in this project are trained from scratch. This makes the comparison more controlled, but it may not produce the best possible performance for every architecture. Some models, especially DeiT-Tiny, may benefit significantly from pretrained weights, stronger data augmentation, learning rate scheduling, or a larger training dataset.

Fifth, Grad-CAM provides useful visual explanations for CNN-based models, but it does not prove that the model truly understands semantic content. A heatmap only shows regions that influence the prediction. It cannot fully explain the internal reasoning of the model, and it may still miss shortcut learning or dataset bias.

In addition, Grad-CAM is not directly applied to DeiT-Tiny in this project. Since DeiT-Tiny is a Transformer-based model, its explanation should be analyzed using attention-based methods such as attention visualization or attention rollout. This remains a limitation of the current explainability analysis.

Finally, the Gradio demo is mainly designed for interactive testing and presentation. It is not a full production deployment system. Additional work would be needed for robust online deployment, model monitoring, input validation, user management, and evaluation on real-world images.

Chapter 10

Conclusion and Future Work

This project builds a complete pipeline for REAL/FAKE image classification using the CIFAKE dataset. The workflow includes dataset analysis, preprocessing, model selection, training setup, training behavior analysis, final test evaluation, Grad-CAM explainability analysis, and an interactive Gradio demo.

Five models are trained and compared: SimpleCNN, ResNet18, EfficientNet-B0, DenseNet121, and DeiT-Tiny. The final test results show that CNN-based models perform better than DeiT-Tiny under the current setup. DenseNet121 achieves the best overall result, with a test accuracy of 95.120

The results suggest that CNN architectures are well suited to CIFAKE classification. This is reasonable because the task depends strongly on local visual evidence such as texture patterns, edges, background irregularities, and artifact-like regions. DenseNet121 performs particularly well because dense feature reuse may help preserve and combine low-level and mid-level visual cues.

Grad-CAM analysis provides additional insight into CNN model behavior. The visual explanations suggest that the CNN-based models may rely not only on semantic object regions, but also on texture, background, edge, and artifact-like areas. This supports the idea that REAL/FAKE image detection is different from ordinary object classification. For DeiT-Tiny, attention-based explanation methods should be explored in future work.

The Gradio demo completes the project by providing an interactive interface for prediction and explanation. It allows users to upload an image, view the predicted label, inspect class probabilities, and observe the Grad-CAM overlay. This makes the project easier to demonstrate and helps connect the experimental results with practical usage.

Future work can extend this project in several directions. First, additional explainability methods such as LIME, SHAP, or Transformer attention rollout can be tested. This would provide more diverse explanations and make the analysis less dependent on Grad-CAM alone.

Second, higher-resolution datasets should be used. Higher-resolution images would provide more detailed visual evidence and allow more meaningful explanation heatmaps.

Third, future experiments should evaluate generalization across different image generators. Instead of using only one synthetic-image source, the model should be tested on images produced by different generative models and different prompt styles.

Fourth, pretrained fine-tuning can be explored. CNN models and Transformer models may achieve better performance when initialized with pretrained weights. DeiT-Tiny in particular may benefit from pretrained initialization, stronger augmentation, learning rate scheduling, and more careful Transformer-oriented training strategies.

Finally, the Gradio demo can be improved and deployed online. A deployed system

would allow users to test images more easily and could serve as a practical demonstration of explainable AI for synthetic image detection.

Overall, the project shows that deep learning models can effectively distinguish REAL and FAKE images in the CIFAKE dataset, and that explainability analysis is important for understanding how these models make predictions.

References

- Bird, J. J., and Lotfi, A. *CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images*, 2023.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017.